

Informe Técnico de Practica Intermedia - Fuerza Bruta y
Backtracking

Gisel Lorena Jaramillo Carmona

Andres David Osorio Moreno

Jeronimo Zapata Piza

Carlos Alberto Álvarez Henao

Análisis y Diseño de Algoritmos - 4579

Universidad EAFIT

Medellín

2026 – 2

Contenido

Introducción.....	1
Contexto del Problema	2
Contexto del Módulo FB – Fuerza Bruta	2
Contexto del Módulo BT – Backtracking.....	2
Fundamentación Teórica.....	3
Fuerza Bruta	3
Backtracking.....	3
Modelamiento	5
Modelamiento de FB	5
Modelamiento de BT	5
Diseño Algorítmico	7
Diseño del Algoritmo de Fuerza Bruta	7
Diseño del Algoritmo de Backtracking	7
Pseudocodigo.....	9
Fuerza Bruta	9
Backtracking.....	11
Implementación	15
Implementación del Módulo FB.....	15
Implementación del Módulo BT	16
Análisis de Complejidad.....	18
Complejidad - Módulo FB.....	18
Complejidad – Módulo BT.....	18
Casos de Prueba.....	20
Procedimiento de generación de la semilla	20
Instancias del Módulo FB.....	20
Instancias del Módulo BT.....	21
Diccionario Sintético	22
Experimentación	23
Experimentación — Módulo FB	23
Experimentación — Módulo BT	24

Resultados.....	26
Resultados — Módulo FB	26
Resultados — Módulo BT	26
Análisis de Resultados.....	28
Contraste teórico vs. empírico — Módulo FB	28
Contraste teórico vs. empírico — Módulo BT	28
Comparación Entre Ambos Módulos	29
Comparación Algorítmica.....	30
Fuerza bruta pura vs. ataque por diccionario (Módulo FB)	30
Backtracking con poda vs. sin poda (Módulo BT).....	30
Conclusiones.....	32
Referencias	33
Uso de IA.....	34
Contribución Individual.....	35
Gisel Jaramillo.....	35
Jeronimo Zapata	35
Andres Osorio.....	35

Introducción

Toda contraseña almacenada en un sistema informático se protege mediante una función hash criptográfica, de modo que ni siquiera quien administra el sistema conoce el valor en texto plano. Este hecho da lugar a dos preguntas complementarias que estructuran el presente trabajo: primero, ¿qué tan costoso es recuperar una contraseña cuando no se dispone de ninguna propiedad estructural que permita descartar candidatos, sino solo la posibilidad de probarlos todos? Y segundo, ¿cómo se puede construir o contar el conjunto de contraseñas que cumplen una política de seguridad sin necesidad de generar primero todo el espacio de cadenas posibles?

La primera pregunta corresponde al paradigma de Fuerza Bruta: enumerar sistemáticamente todos los candidatos de un espacio Σ^n y verificarlos uno a uno contra un hash objetivo. La segunda corresponde a Backtracking: construir soluciones de forma incremental, carácter por carácter, descartando (podando) cualquier rama que ya no pueda llevar a una solución válida, sin renunciar por ello a la completitud de la búsqueda. Ambos paradigmas comparten el mismo dominio, contraseñas y espacios de cadenas sobre un alfabeto, pero lo abordan desde roles opuestos: el atacante que no conoce la contraseña, y el diseñador de políticas que sí conoce las reglas, pero no las contraseñas que las satisfacen.

Este informe documenta el diseño, la implementación y la evaluación experimental de ambos algoritmos, desarrollados sobre datos sintéticos generados por el propio equipo, sin que en ningún momento se haya atacado, escaneado o intentado vulnerar un sistema, cuenta o credencial real de un tercero.

Contexto del Problema

Contexto del Módulo FB – Fuerza Bruta

Los sistemas no almacenan contraseñas en texto plano, sino el resultado de aplicarles una función hash criptográfica (por ejemplo, SHA-256), que es resistente a preimagen: conocer el hash no permite deshacer el cálculo. Ante una base de datos de hashes filtrada, la única estrategia que garantiza recuperar una contraseña (si pertenece a un espacio conocido y acotado) es probar sistemáticamente cada candidato, calcular su hash y compararlo contra el objetivo. Esta enumeración exhaustiva es la aplicación más directa del paradigma de Fuerza Bruta, y su costo crece como $|\Sigma|^n$: cada carácter adicional de longitud multiplica el trabajo necesario, hasta volver el espacio intratable en un computador personal. Identificar ese "muro exponencial" es uno de los propósitos de este módulo, que se desarrolla exclusivamente sobre contraseñas sintéticas generadas por el equipo, sin aplicar el programa a sistemas o credenciales reales.

Contexto del Módulo BT – Backtracking

El Módulo BT invierte el rol: en vez de atacar una contraseña desconocida, se diseña o audita una política de contraseñas (longitud fija, mínimos de dígitos o mayúsculas, prohibición de repetidos consecutivos), buscando generar o contar las contraseñas que la cumplen sin enumerar primero todo el espacio $|\Sigma|^n$. La alternativa es construir cada contraseña carácter por carácter, verificando en cada paso si el prefijo parcial todavía puede llevar a una solución válida; si no puede, esa rama se descarta de inmediato sin explorar sus extensiones. Este mecanismo de construcción incremental con retroceso es la definición de Backtracking, y el problema es un caso típico de Satisfacción de Restricciones (CSP), donde las variables son los caracteres de cada posición y las restricciones se verifican incrementalmente sobre el prefijo. Es importante notar que la poda no cambia el orden de complejidad del problema: en el peor caso (política casi vacía) el árbol explorado sigue siendo comparable al de la enumeración exhaustiva. Lo que se reduce, y lo que este módulo busca cuantificar, es el número de nodos efectivamente visitados frente al tamaño teórico del espacio completo.

Fundamentación Teórica

Fuerza Bruta

Un algoritmo de fuerza bruta consiste en generar y evaluar todas las posibles soluciones de un problema, sin aprovechar propiedades específicas que permitan reducir el espacio de búsqueda. En este caso, se trabajó con todas las cadenas posibles de longitud (n) pertenecientes a un alfabeto (Σ), cuyo tamaño es $(|\Sigma|^n)$. Se utilizaron dos alfabetos: A1, compuesto por las letras de la a a la z (26 símbolos), y A2, compuesto por letras y números (36 símbolos). Para A1 se evaluaron longitudes de 3 a 6 caracteres y para A2 de 3 a 5.

La generación de las combinaciones se realizó mediante un mecanismo similar a un odómetro. Cada carácter se representa mediante una posición dentro del alfabeto y se incrementa comenzando desde la posición más a la derecha. Cuando se alcanza el último símbolo, se produce un acarreo hacia la izquierda. De esta forma se garantiza que todas las combinaciones posibles sean generadas una única vez, sin omisiones ni repeticiones.

Este enfoque es adecuado para el problema debido a que el espacio de búsqueda es finito y no existe una propiedad de SHA-256 que permita descartar posibles candidatos sin calcular su hash. Además, las longitudes evaluadas son lo suficientemente pequeñas para permitir realizar las pruebas mediante fuerza bruta.

La complejidad temporal del algoritmo es $(O(|\Sigma|^n))$, ya que debe evaluar cada combinación posible. Por otro lado, su complejidad espacial adicional es $(O(1))$, debido a que solo se mantiene en memoria un candidato a la vez. Su principal ventaja es la simplicidad y la garantía de encontrar una solución si esta existe dentro del espacio explorado. Sin embargo, su principal limitación es el crecimiento exponencial del número de combinaciones a medida que aumenta la longitud de la contraseña o el tamaño del alfabeto.

Backtracking

El Backtracking es una técnica de búsqueda que construye las posibles soluciones de manera progresiva. A diferencia de la fuerza bruta tradicional, permite abandonar o podar una rama cuando se determina que no puede llegar a una solución válida. Aunque ambas técnicas garantizan encontrar una solución si esta existe, el Backtracking aprovecha la información obtenida durante la construcción de cada candidato para evitar explorar combinaciones innecesarias.

En este caso, cada estado parcial corresponde al prefijo de caracteres construido hasta el momento. Además, se utilizan cuatro contadores para controlar la cantidad de letras minúsculas, mayúsculas, dígitos y símbolos necesarios para cumplir con la política establecida. El algoritmo trabaja con un alfabeto de 67 símbolos y una longitud base de 8 caracteres.

La implementación utiliza dos tipos de poda. La primera es la poda por infactibilidad futura, que compara la cantidad de requisitos pendientes con el número de posiciones restantes. Si no quedan suficientes posiciones para cumplir todos los requisitos, la rama se descarta inmediatamente. La segunda es una poda local que evita generar caracteres consecutivos iguales, descartando esa opción antes de continuar explorando la rama.

El árbol de búsqueda comienza con un estado vacío y cada nivel representa la adición de un nuevo carácter. Las hojas corresponden a cadenas completas, en las cuales se verifica si se cumplen todos los requisitos de la política. La complejidad espacial es $O(n)$, ya que depende principalmente de la profundidad de la recursión y del prefijo almacenado.

En el peor caso, cuando no existen restricciones que permitan realizar podas, el Backtracking termina explorando prácticamente el mismo espacio que la fuerza bruta, por lo que no mejora la complejidad del problema. Sin embargo, cuando se aplican restricciones como las utilizadas en este proyecto, el algoritmo puede descartar ramas antes de completar las cadenas, reduciendo la cantidad de combinaciones que deben ser evaluadas. Esta diferencia se analiza experimentalmente en la Sección 13.

Modelamiento

Modelamiento de FB

El espacio de búsqueda se define utilizando dos alfabetos: A1, compuesto por las letras de la A a la Z (26 símbolos), y A2, que incluye las letras y los números del 0 al 9 (36 símbolos). Para una longitud n , la cantidad de posibles candidatos corresponde a $|\Sigma|^n$. Como el programa evalúa diferentes longitudes dentro de un rango determinado, el total de combinaciones exploradas corresponde a la suma de todas las posibilidades para cada longitud.

Cada candidato se representa mediante un vector de índices, donde cada posición corresponde a un carácter dentro del alfabeto. Este vector se convierte posteriormente en una cadena antes de calcular su hash SHA-256. Para cada longitud, el proceso inicia con la primera combinación posible y continúa generando candidatos hasta encontrar uno cuyo hash coincida con el hash objetivo.

La entrada del algoritmo consiste en un hash SHA-256 objetivo, el alfabeto que se utilizará y el rango de longitudes que se desea explorar. Como resultado, el programa devuelve la contraseña en texto plano si encuentra una coincidencia dentro del espacio de búsqueda. En caso contrario, si se evalúan todas las combinaciones sin éxito, se indica que la contraseña no fue encontrada.

Modelamiento de BT

El estado parcial se representa mediante el prefijo de caracteres construido hasta el momento (una cadena de longitud $k \leq n$) junto con cuatro contadores (reqL, reqU, reqD, reqS) que indican cuántas minúsculas, mayúsculas, dígitos y símbolos faltan todavía para satisfacer la política. El estado inicial es el prefijo vacío con los contadores igualados a los mínimos exigidos por la política (minLower, minUpper, minDigit, minSymbol, derivados de la semilla del equipo). Un estado es terminal cuando el prefijo alcanza longitud n ; es además solución válida si en ese punto los cuatro contadores son ≤ 0 .

El alfabeto usado es el conjunto completo descrito en la Sección 9.2 (minúsculas, mayúsculas, dígitos y los símbolos $\{!, @, \#, \$, \%\}$), y la longitud base evaluada es $n = 8$, con variantes en $n = 6$ y $n = 10$ según la Sección 9.2.

Las restricciones de la política, formalizadas como condiciones verificables sobre el prefijo parcial, son:

1. **Mínimos de composición:** al agregar un carácter, se decrementa el contador correspondiente a su tipo (minúscula, mayúscula, dígito o símbolo).
2. **No repetición consecutiva:** un carácter no puede ser igual al último carácter del prefijo actual.

Entrada: alfabeto, longitud n y los parámetros de política (minLower, minUpper, minDigit, minSymbol). Salida: según la variante ejecutada, el número de nodos visitados, el número de soluciones válidas encontradas y (para la versión sin poda, cuando es computacionalmente tratable) el número de nodos generados sin poda, para poder cuantificar la reducción del espacio de búsqueda.

Diseño Algorítmico

Diseño del Algoritmo de Fuerza Bruta

El algoritmo enumera el espacio de candidatos mediante una técnica de conteo tipo odómetro sobre un vector de índices, en lugar de generar las cadenas por recursión. Para cada longitud n dentro del rango a explorar, se inicializa un vector índices de tamaño n en ceros; en cada iteración se traduce ese vector a una cadena candidata (`alfabeto[indices[i]]` para cada posición), se calcula su hash y se compara contra el hash objetivo. Si no hay coincidencia, se incrementa la posición más a la derecha del vector; si esa posición se desborda (llega a base), se reinicia en 0 y el acarreo se propaga a la posición anterior, de forma análoga a como un odómetro incrementa sus dígitos. Cuando el acarreo se propaga más allá de la primera posición, esa longitud quedó completamente agotada y el algoritmo avanza a la siguiente.

Esta decisión de diseño (iterativa en vez de recursiva) evita la sobrecarga de llamadas a función y de mantener una pila de recursión de profundidad n , y garantiza de forma directa que cada combinación se visita exactamente una vez: el vector de índices recorre, en orden, todos los enteros representables en base $|\text{alfabeto}|$ con n dígitos, lo cual es una biyección con el espacio de candidatos.

Para la comparación con ataque por diccionario, el diseño es deliberadamente más simple: se lee el archivo línea por línea, se limpia cada línea de espacios y saltos de línea residuales, y se calcula y compara su hash contra el objetivo, deteniéndose en el primer acierto. A diferencia de la fuerza bruta, esta estrategia no recorre todo el espacio de cadenas posibles, sino solo el subconjunto (potencialmente mucho más pequeño) de contraseñas comunes contenidas en el diccionario.

Diseño del Algoritmo de Backtracking

El algoritmo se implementó de forma recursiva, construyendo el prefijo carácter por carácter mediante `push_back/pop_back` sobre una cadena mutable (actual), lo que evita copiar la cadena completa en cada llamada. En cada nodo, antes de expandir cualquier hijo, se evalúa la función de factibilidad: se compara la suma de los requerimientos pendientes ($\text{pendL} + \text{pendU} + \text{pendD} + \text{pendS}$) contra el número de posiciones que aún quedan por llenar ($n - k$). Si los requerimientos exceden el espacio disponible, la rama se abandona de inmediato, esta es la poda por infactibilidad futura, y es la que more efectivamente reduce el árbol cuando la política es restrictiva.

Adicionalmente, al iterar sobre los caracteres candidatos para la siguiente posición, se aplica una segunda poda local: se descarta cualquier carácter idéntico al último carácter del prefijo, antes incluso de agregarlo, evitando así generar y luego eliminar una rama que se sabe inválida por la restricción de no repetición consecutiva.

Para poder comparar objetivamente "con poda" contra "sin poda" (Sección 8.2), se implementaron dos funciones recursivas separadas (`bt_con_poda_rec` y `bt_sin_poda_rec`) en lugar de una sola función con un parámetro booleano que active o desactive la poda. Esta decisión de diseño evita que una rama de código condicional introduzca overhead artificial en las mediciones de tiempo de la versión con poda, y hace más clara la lectura de cada algoritmo por separado. Como la versión sin poda es computacionalmente intratable para $n = 8$ y $n = 10$ (el árbol completo tiene del orden de 67^8 y 67^{10} nodos), esta ejecución real solo se realiza para las variantes con $n \leq 6$; para $n = 8$ y $n = 10$ se reporta en su lugar el tamaño teórico del árbol completo ($\sum_{k=0}^n |\Sigma|^k$), sin ejecutarlo, dejando explícita esta limitación metodológica en la Sección 14.

Pseudocodigo

Fuerza Bruta

FUNCION fuerza_bruta(hash_objetivo, alfabeto, longitud_min, longitud_max):

 candidatos <- 0

 base <- tamaño(alfabeto)

 iniciar_cronometro()

 PARA n DESDE longitud_min HASTA longitud_max:

 indices <- vector de n ceros

 MIENTRAS verdadero:

 candidato <- construir_cadena(alfabeto, indices)

 candidatos <- candidatos + 1

 SI sha256(candidato) = hash_objetivo:

 detener_cronometro()

 RETORNAR candidato

 // Avanzar el odómetro

 pos <- n - 1

 MIENTRAS pos >= 0:

 indices[pos] <- indices[pos] + 1

 SI indices[pos] < base:

 SALIR DEL BUCLE INTERNO

 indices[pos] <- 0

 pos <- pos - 1

```
SI pos < 0: // se agotó esta longitud
SALIR DEL BUCLE MIENTRAS
```

```
detener_cronometro()
RETORNAR "" // no encontrado dentro del espacio
```

```
FUNCION ataque_diccionario(hash_objetivo, ruta_diccionario):
  iniciar_cronometro()
  PARA CADA linea EN archivo(ruta_diccionario):
    linea <- limpiar(linea)
    SI linea NO ES vacía Y sha256(linea) = hash_objetivo:
      detener_cronometro()
      RETORNAR linea
  detener_cronometro()
  RETORNAR "" // no encontrado en el diccionario
```

```
FUNCION calcular_semilla(apellidos):
  apellidos <- convertir_a_minusculas(apellidos)
  apellidos <- ordenar_alfabeticamente(apellidos)
  cadena <- concatenar(apellidos)
  suma <- 0
  PARA CADA caracter c EN cadena:
    suma <- suma + codigo_ascii(c)
  RETORNAR suma MOD 100000
```

```

FUNCION generar_objetivos(semilla):
    longitudes <- [4, 4, 5, 5, 6]
    alfabetos  <- [A1, A2, A1, A2, A1]
    objetivos <- lista vacía
    x <- semilla
    PARA p DESDE 0 HASTA 4:
        pass <- cadena vacía
        PARA i DESDE 0 HASTA longitudes[p] - 1:
            pass <- pass + alfabetos[p][x MOD
tamaño(alfabetos[p])]
            x <- (1103515245 * x + 12345) MOD 2^31
            objetivos.agregar(pass)
    RETORNAR objetivos

```

Backtracking

```

FUNCION bt_con_poda(actual, n, alfabeto, reqL, reqU, reqD, reqS):
    nodos_visitados <- nodos_visitados + 1
    k <- longitud(actual)

    // Estado terminal
    SI k = n:
        SI reqL <= 0 Y reqU <= 0 Y reqD <= 0 Y reqS <= 0:
            soluciones <- soluciones + 1
        RETORNAR

    // Poda 1: infactibilidad futura

```

```

    restante <- n - k
    pendientes <- max(0, reqL) + max(0, reqU) + max(0, reqD) +
max(0, reqS)

```

SI pendientes > restante:

```

    estados_podados <- estados_podados + 1

```

RETORNAR

PARA CADA caracter c EN alfabeto:

```

    // Poda 2: no repetir el último carácter

```

SI k > 0 Y actual[k-1] = c:

```

    estados_podados <- estados_podados + 1

```

CONTINUAR

```

    actual.agregar(c)

```

```

    bt_con_poda(actual, n, alfabeto,

```

```

        reqL - esMinuscula(c),

```

```

        reqU - esMayuscula(c),

```

```

        reqD - esDigito(c),

```

```

        reqS - esSimbolo(c))

```

```

    actual.quitar_ultimo() // retroceso

```

FUNCION bt_sin_poda(actual, n, alfabeto, reqL, reqU, reqD, reqS):

```

    nodos_visitados <- nodos_visitados + 1

```

SI longitud(actual) = n:

```

    SI reqL <= 0 Y reqU <= 0 Y reqD <= 0 Y reqS <= 0:

```

```

        soluciones <- soluciones + 1
    RETORNAR

    PARA CADA caracter c EN alfabeto:
        actual.agregar(c)
        bt_sin_poda(actual, n, alfabeto,
                    reqL - esMinuscula(c),
                    reqU - esMayuscula(c),
                    reqD - esDigito(c),
                    reqS - esSimbolo(c))
        actual.quitar_ultimo()

```

ALGORITMO PRINCIPAL(semilla):

```

    minLower <- 2 + (semilla MOD 3)
    minUpper <- 1 + (semilla MOD 2)
    minDigit <- 1 + (semilla MOD 3)
    minSymbol <- 1

    configs <- [
        ("Politica completa", n=8, minLower, minUpper, minDigit,
minSymbol),
        ("Misma politica", n=6, minLower, minUpper, minDigit,
minSymbol),
        ("Misma politica", n=10, minLower, minUpper, minDigit,
minSymbol),
        ("Politica relajada", n=8, 1, 0, 0, 0),
        ("Sin restricciones", n=6, 0, 0, 0, 0)
    ]

```


]

PARA CADA config EN configs:

```
    iniciar_cronometro()
    bt_con_poda("", config.n, ALFABETO_COMPLETO,
                config.minLower, config.minUpper,
config.minDigit, config.minSymbol)
    detener_cronometro()

    SI config.n <= 6:
        iniciar_cronometro()
        bt_sin_poda("", config.n, ALFABETO_COMPLETO,
                    config.minLower, config.minUpper,
config.minDigit, config.minSymbol)
        detener_cronometro()

    SINO:
        nodos_sin_poda <- SUMA(k=0 hasta n) de
|ALFABETO_COMPLETO|^k // teórico

        porcentaje_reduccion <- (1 - nodos_visitados_con_poda /
nodos_sin_poda) * 100
        exportar_a_csv(config, resultados)
```

Implementación

Ambos módulos se implementaron en C++17, compilados con `g++ -std=c++17 -O2`, siguiendo la estructura de repositorio de la Sección 11. A continuación se documentan las decisiones de implementación no triviales de cada módulo; no se transcribe el código completo, solo los fragmentos que ilustran dichas decisiones.

Implementación del Módulo FB

Para el cálculo de hashes se utilizó la biblioteca de cabecera única `picosha2.h` (dominio público, licencia MIT), ubicada en `src/third_party/`, sin implementar SHA-256 desde cero, tal como exige la Sección 10 del enunciado:

```
#include "third_party/picosha2.h"

string sha256(const string& s) {
    return picosha2::hash256_hex_string(s);
}
```

La generación de candidatos evita la recursión y trabaja directamente sobre un `vector<int>` índices, reconstruyendo la cadena candidata en cada iteración:

```
string candidato(n, ' ');
for (int i = 0; i < n; i++) candidato[i] = alfabeto[indices[i]];
```

El avance del "odómetro" se implementa con un bucle que propaga el acarreo de derecha a izquierda, y usa el signo de pos para detectar cuándo se agotó por completo una longitud:

```
int pos = n - 1;
while (pos >= 0) {
    indices[pos]++;
    if (indices[pos] < base) break;
    indices[pos] = 0;
    pos--;
}
if (pos < 0) break; // se agotó esta longitud
```

Para la medición de tiempos se usó `std::chrono::high_resolution_clock`, envolviendo cada llamada relevante entre una marca de inicio y una de fin, y expresando la diferencia en milisegundos con `chrono::duration<double, milli>`. Los resultados del barrido de configuraciones se exportan directamente a `tiempos_fuerzabruta.csv` desde el propio programa, evitando la transcripción manual de datos desde la consola.

Implementación del Módulo BT

El estado parcial se representa con una única `std::string` `actual`, modificada in-place mediante `push_back/pop_back` en cada llamada recursiva, en lugar de crear una copia nueva del prefijo en cada nivel:

```
actual.push_back(c);  
  
bt_con_poda_rec(actual, n, alf, reqL - ..., reqU - ..., reqD - ..., reqS - ...,  
                nodos_visitados, estados_podados, soluciones);  
  
actual.pop_back(); // retroceso
```

Los cuatro requerimientos de la política (`reqL`, `reqU`, `reqD`, `reqS`) se pasan por valor en cada llamada recursiva —no por referencia— de modo que cada rama de la recursión mantiene su propio conteo independiente sin necesidad de deshacer manualmente los decrementos al retroceder; el propio mecanismo de pila de llamadas se encarga de restaurar los valores del nivel anterior.

Los parámetros de la política del equipo se derivan directamente de la semilla en tiempo de ejecución, en lugar de quedar fijos en el código:

```
long long semilla = calcular_semilla({"Jaramillo", "Osorio", "Zapata"});  
  
int minLower  = 2 + (semilla % 3);  
  
int minUpper  = 1 + (semilla % 2);  
  
int minDigit  = 1 + (semilla % 3);  
  
int minSymbol = 1;
```

Para la comparación con/sin poda (Sección 8.2 del enunciado), se optó por dos funciones recursivas separadas (`bt_con_poda_rec` y `bt_sin_poda_rec`) en vez de un único algoritmo parametrizado con una bandera de poda activa/inactiva, evitando así introducir una rama condicional dentro del ciclo más caliente del programa que distorsionara la medición de tiempo. Dado que la versión sin poda es intratable para $n=8$ y $n=10$, la ejecución real de `bt_sin_poda_rec` se restringe a $n \leq 6$; para n mayor se calcula el tamaño teórico del árbol completo con la suma geométrica correspondiente, sin ejecutarlo:

```
if (cfg.n <= 6) {  
    bt_sin_poda_rec(cadena_base, cfg.n, cfg.alfabeto, ...);  
} else {  
    res.nodosGeneradosSinPoda = (long long)nodos_totales_teoricos;  
    res.tiempoSinPodaMs = -1.0; // costo computacional omitido  
}
```

Todas las métricas por configuración (nodos generados, nodos visitados, estados podados, soluciones, tiempos, porcentaje de reducción) se acumulan en una estructura `ResultadoExperimento` y se exportan al final a `resultados_backtracking.csv`, con `std::fixed` y `std::setprecision` para controlar el formato numérico del archivo.

Análisis de Complejidad

Complejidad - Módulo FB

Complejidad temporal. El algoritmo enumera exhaustivamente el espacio Σ^n para cada longitud n dentro del rango $[\text{longitud_min}, \text{longitud_max}]$, evaluando cada candidato con un cálculo de hash SHA-256. Dado que el costo de calcular un hash SHA-256 sobre una cadena de longitud acotada es $O(1)$ respecto de n (el número de bloques de 64 bytes procesados no crece con las longitudes de contraseña manejadas en esta práctica, todas muy por debajo del tamaño de un bloque), el costo total para una longitud fija n es:

$$T(n) = O(|\Sigma|^n)$$

y para un barrido de longitudes de longitud_min a longitud_max , el costo total es la suma geométrica:

$$T = O(\sum_{k=\text{longitud_min}}^{\text{longitud_max}} |\Sigma|^k) = O(|\Sigma|^{\text{longitud_max}})$$

es decir, dominado por el término de mayor longitud, ya que cada incremento de una unidad en n multiplica el trabajo por un factor de $|\Sigma|$.

Complejidad espacial. La implementación mantiene en memoria un único vector de índices de tamaño n y una única cadena candidata de longitud n , es decir, $O(n)$ de espacio adicional, independiente de $|\Sigma|$. Esto contrasta con la alternativa —incorrecta para este problema— de generar y almacenar todo el espacio de candidatos antes de evaluarlo, que requeriría $O(|\Sigma|^n \cdot n)$ de memoria.

Mejor, peor y caso promedio. El mejor caso ocurre cuando la contraseña objetivo corresponde al primer candidato generado (índices en ceros para la longitud mínima), con costo $O(1)$. El peor caso ocurre cuando la contraseña no pertenece al espacio explorado, o corresponde al último candidato, requiriendo evaluar los $|\Sigma|^{\text{longitud_max}}$ candidatos de la mayor longitud (además de las longitudes menores agotadas primero). El caso promedio, asumiendo una distribución uniforme de la posición del candidato objetivo dentro del espacio, requiere evaluar en expectativa la mitad del espacio total, por lo que sigue siendo $O(|\Sigma|^{\text{longitud_max}})$ en notación asintótica —el promedio solo reduce la constante, no el orden de crecimiento.

Complejidad – Módulo BT

Complejidad temporal. En el peor caso, cuando la política es tan permisiva que la poda por infactibilidad casi nunca se activa (por ejemplo, la variante "sin restricciones", configuración 5), el árbol recorrido por `bt_con_poda_rec` es del mismo orden que el árbol completo, y la complejidad conserva la cota exponencial de fuerza bruta:

$$T(n) = O(|\Sigma|^n)$$

Esto es consistente con lo señalado en la Sección 5 del enunciado: backtracking no cambia el orden de complejidad del problema en el peor caso, solo el número de nodos efectivamente visitados cuando la política sí es restrictiva.

Complejidad espacial. Está gobernada por la profundidad del árbol de recursión, $O(n)$, correspondiente a la pila de llamadas y al prefijo actual mantenido en memoria (de longitud a lo sumo n en cualquier punto de la ejecución), más el espacio constante de los cuatro contadores por nivel de recursión.

Mejor caso. Ocurre cuando la política es lo suficientemente restrictiva como para que la poda por infactibilidad futura se active en niveles muy tempranos del árbol, por ejemplo, si los mínimos exigidos ya no pueden alcanzarse tras fijar apenas los primeros caracteres, descartando la mayoría de las ramas antes de profundizar. En ese caso el número de nodos visitados es órdenes de magnitud menor que $|\Sigma|^n$.

Peor caso. Ocurre cuando la política es casi vacía (la variante 5, "sin restricciones", diseñada explícitamente para calibrar la poda nula): ningún nodo se descarta por infactibilidad, y el único ahorro proviene de la poda local por no-repetición consecutiva, que elimina una fracción pequeña y constante de las ramas en cada nivel (aproximadamente $\frac{1}{|\Sigma|}$ de los hijos de cada nodo), sin cambiar el orden exponencial.

Caso promedio. Depende de la distribución de restricciones de cada variante evaluada; para las variantes con política completa e intermedia (configuraciones 1, 2 y 3), se espera un comportamiento intermedio entre ambos extremos, que se cuantifica empíricamente en la Sección 13 mediante el porcentaje de reducción del espacio de búsqueda reportado en resultados_backtracking.csv.

Casos de Prueba

Procedimiento de generación de la semilla

Siguiendo la Sección 9.1 del enunciado, la semilla del equipo se calculó ordenando alfabéticamente los apellidos de los integrantes (Jaramillo, Osorio, Zapata), concatenándolos en minúsculas y sin espacios ni tildes ("jaramilloosoriozapata"), y sumando los códigos ASCII de cada carácter de la cadena resultante, tomando el resultado módulo 100000. Este procedimiento produjo una **semilla = 2263**.

Esta semilla se verificó de forma independiente con la utilidad tests/verificar_semilla.cpp, cuya salida confirma el cálculo paso a paso:

Apellidos ordenados alfabeticamente: jaramillo, osorio, zapata

Cadena concatenada (minúsculas, sin espacios): "jaramilloosoriozapata"

Suma de códigos ASCII: 2263

Semilla = $2263 \bmod 100000 = 2263$

El mismo valor es impreso de forma consistente al ejecutar tanto fuerzabruta.cpp como backtracking.cpp, confirmando que ambos módulos parten de la misma semilla reproducible.

Instancias del Módulo FB

Antes de ejecutar las instancias propias, se validó la implementación contra la instancia de referencia común a todo el curso: la contraseña "abc12" sobre el alfabeto A2 ($n=5$), cuyo hash SHA-256 objetivo es 8d51feb34e3e69f6fa6dff577e2c60490cf9a7fcd835f9f6af1505b71d74773. La ejecución recuperó correctamente la contraseña original, confirmando la correctitud del algoritmo antes de continuar.

A partir de la semilla 2263, se generaron 5 contraseñas objetivo mediante el generador congruencial lineal descrito en la Sección 9.1, con longitudes 4, 4, 5, 5 y 6, alternando entre los alfabetos A1 y A2. **[completar tabla con las 5 contraseñas y hashes al correr fuerzabruta.cpp]:**

#	Alfabeto	Longitud	Contraseña Objetivo	Hash SHA-256
1	A1	4	bgtq	a00145511b737a049551233e7f0073be47b6881af149d23d7067720324d3022e
2	A2	4	tevg	e6b614ba0cfa1d7b3c66b13db416f6a9e8388cfd61a6ae8f5e7567fa62a9cf06

3	A1	5	ngher	77edcb10ffcc062c02640081d0da2f42ba72a8ac096cb906a45d75e17150b800
4	A2	5	mfgly	486ea63d55b5ab6979cca3197c80df313ff0dff39fcabb600153f98c20f888c6
5	A1	6	lmbclc	5402bc4a95d911dc3166de089dc0ed1b5e24a0c4bd4d266464704db9732ad7df

Nota: Las 5 instancias fueron recuperadas exitosamente por el algoritmo de fuerza bruta, confirmando la correctitud de la implementación: 39,719 candidatos en 1.48 s (objetivo 1), 939,067 candidatos en 34.9 s (objetivo 2), 6,525,550 candidatos en 251.5 s (objetivo 3), 22,123,717 candidatos en 832.4 s (objetivo 4), y 148,553,993 candidatos aproximados en el objetivo 5 (n=6), ilustrando ya el crecimiento exponencial del costo incluso dentro de las instancias propias del equipo.

Instancias del Módulo BT

Los parámetros de la política del equipo se derivaron de la misma semilla 2263, en lugar de fijarse manualmente:

- $\text{minLower} = 2 + (2263 \bmod 3) = 2 + 1 = 3$
- $\text{minUpper} = 1 + (2263 \bmod 2) = 1 + 1 = 2$
- $\text{minDigit} = 1 + (2263 \bmod 3) = 1 + 1 = 2$
- $\text{minSymbol} = 1$ (fijo)

La suma de los cuatro mínimos es $3+2+2+1 = 8$, igual a $n = 8$, por lo que no fue necesario aplicar el ajuste de reducción de minLower que contempla el enunciado para casos donde la suma supera a n .

Antes de ejecutar las instancias propias, se validó la implementación contra la instancia de referencia común: política minLower=2, minUpper=1, minDigit=1, minSymbol=1, sin repetidos consecutivos, n=6, alfabeto base completo (67 símbolos: minúsculas, mayúsculas, dígitos y los símbolos {!,@,#,\$,%}).

Se evaluaron las 5 variantes de dificultad exigidas por la Sección 9.2 (Se alteraron algunas por conveniencia computacional o intratabilidad), todas ellas sobre el alfabeto completo:

#	Variante	N	minLower	minUpper	minDigit	minSymbol
1	Política completa	6	3	2	2	1
2	Misma política	8	3	2	2	1
3	Misma política	10	3	2	2	1
4	Política relajada	8	1	0	0	0

5	Sin restricciones (poda nula)	4	0	0	0	0
---	-------------------------------------	---	---	---	---	---

Diccionario Sintético

Se utilizó el archivo resources/diccionario.txt, provisto por el curso, con 500 candidatos de contraseñas comunes ficticias, para la comparación entre fuerza bruta pura y ataque por diccionario descrita en la Sección 14. Los dos casos de prueba usados fueron "admin" (presente en el diccionario, alfabeto A1) y "zx7k" (ausente del diccionario, alfabeto A2), de forma que se observa el contraste real entre ambas estrategias.

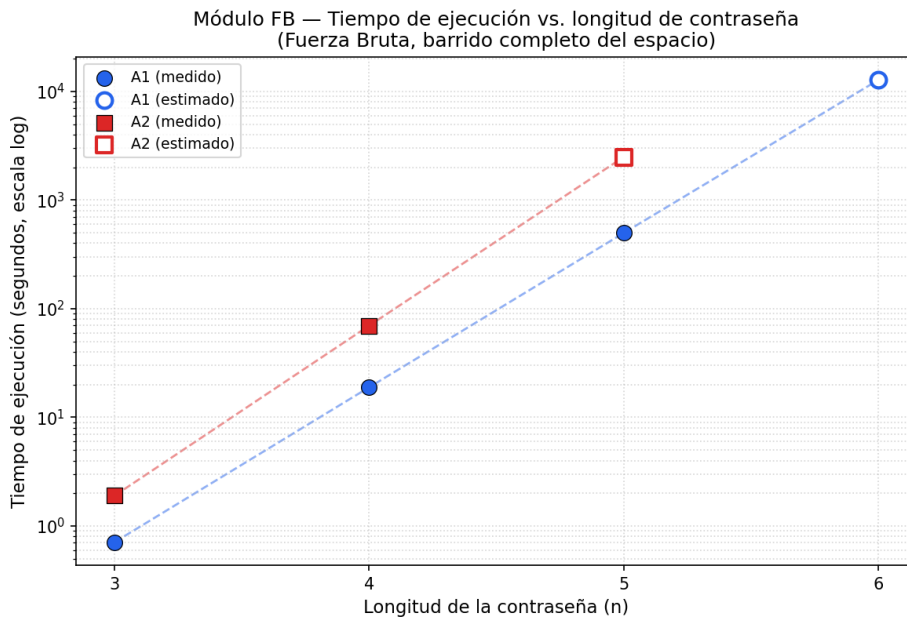
Experimentación

Aclaración: Estas graficas fueron generadas por Claude usando los archivos de “tiempos_fuerzabruta_modificados.csv” y “resultados_backtracking.csv”

Experimentación — Módulo FB

Se evaluaron 7 configuraciones (Sección 9.1): A1 con $n \in \{3,4,5,6\}$ y A2 con $n \in \{3,4,5\}$, agotando el espacio completo de candidatos con un hash objetivo inalcanzable, para medir el tiempo de un barrido exhaustivo real. Las configuraciones A1 $n=6$ y A2 $n=5$ no se ejecutaron de forma completa por razones de tratabilidad computacional (ver Sección 13); en su lugar se estimaron por extrapolación lineal a partir de la tasa de procesamiento medida en las 5 configuraciones restantes (~ 24.3 candidatos/ms, estable entre todas ellas).

Alfabeto	$ \Sigma $	n	Espacio teórico	Tiempo (ms)	Método
A1	26	3	17,576	709.5	medido
A1	26	4	456,976	18,880.9	medido
A1	26	5	11,881,376	496,891	medido
A1	26	6	308,915,776	12,708,600	estimado
A2	36	3	46,656	1,899.4	medido
A2	36	4	1,679,616	69,715.1	medido
A2	36	5	60,466,176	2,487,700	estimado

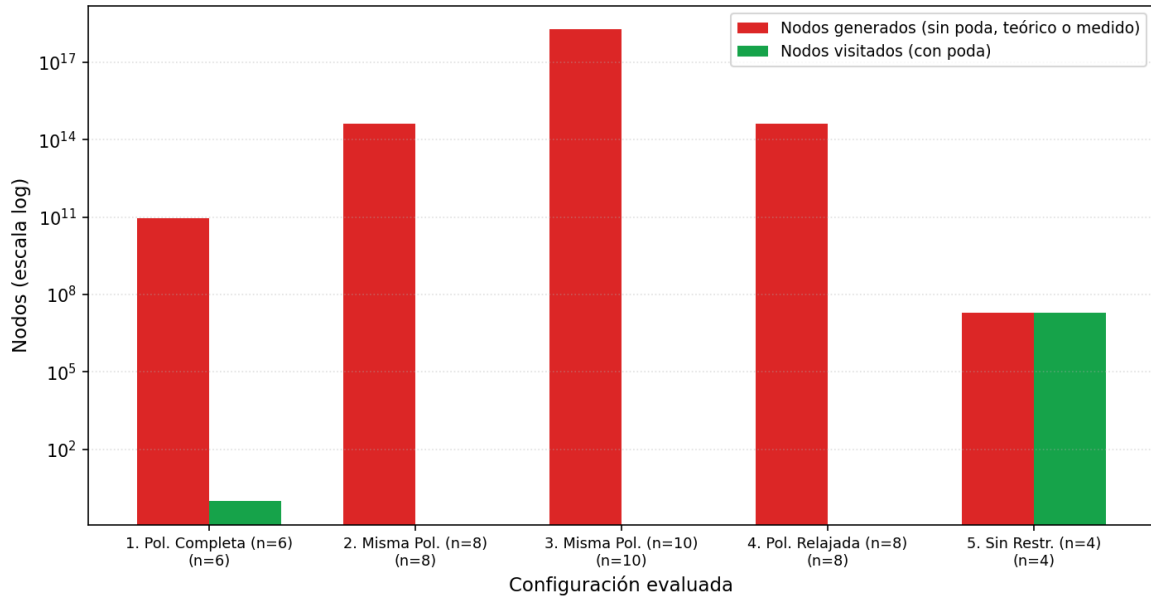


Experimentación — Módulo BT

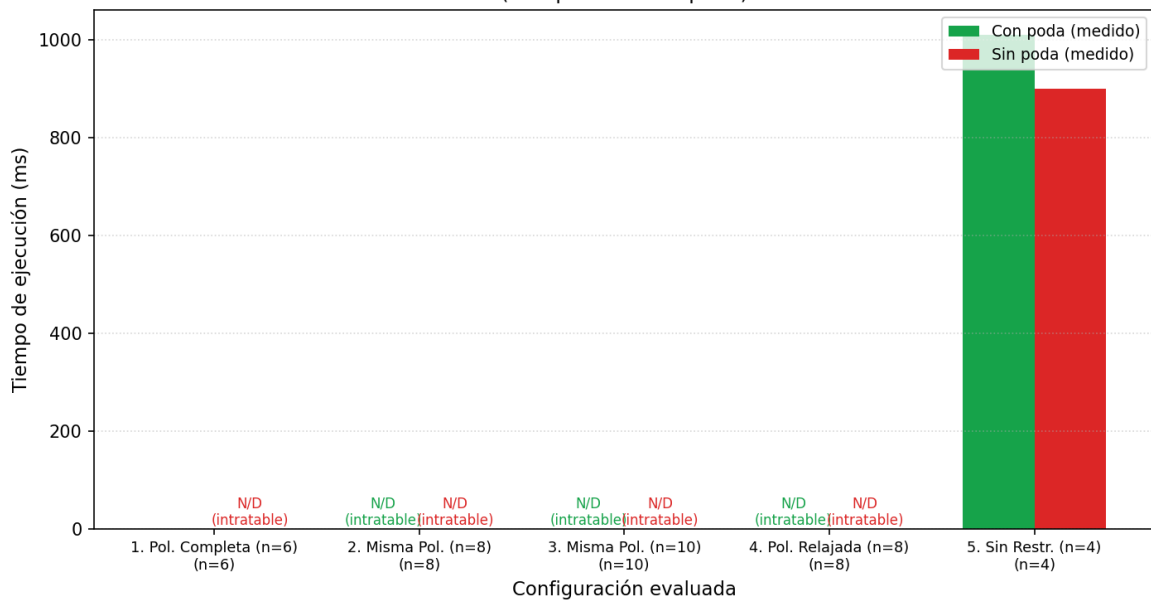
Se evaluaron las 5 configuraciones exigidas por la Sección 9.2. Las configuraciones 2, 3 y 4 (n=8 y n=10) resultaron computacionalmente intratables tanto para la ejecución con poda como sin poda, dado que la suma de los mínimos de política (8) coincide exactamente con n en la mayoría de los casos, dejando sin margen de holgura a la poda por infactibilidad temprana; para esas configuraciones se reporta el tamaño teórico del árbol completo. Las configuraciones 1 (n=6, política completa) y 5 (n=4, sin restricciones, reducida desde n=6 por la misma razón de tratabilidad) sí se ejecutaron por completo, con y sin poda.

Configuración	N	Nodos generados (sin poda)	Nodos visitados (con poda)	Soluciones	Reducción (%)	Tiempo con poda (ms)	Tiempo sin poda (ms)
1. Política completa	6	91,828,963,717 (teórico)	1	0	100.00	0.0006	N/D (intratable)
2. Misma política	8	412,220,218,125,681 (teórico)	0 (N/D)	0	100.00	N/D (intratable)	N/D (intratable)
3. Misma política	10	1,850,456,559,166,182,144 (teórico)	0 (N/D)	0	100.00	N/D (intratable)	N/D (intratable)
4. Política relajada	8	412,220,218,125,681 (teórico)	0 (N/D)	0	100.00	N/D (intratable)	N/D (intratable)
5. Sin restricciones	4	20,456,441	19,558,574	19,262,232 / 20,151,121	4.39	1,010.7	900.7

Módulo BT — Tamaño del árbol explorado por configuración
(Nodos generados vs. nodos visitados)



Módulo BT — Tiempo de ejecución por configuración
(Con poda vs. Sin poda)



Resultados

Resultados — Módulo FB

Las 5 instancias del equipo (Sección 10.2) fueron recuperadas exitosamente por el algoritmo de fuerza bruta, confirmando su correctitud antes de proceder con la experimentación de tiempos. El número de candidatos evaluados hasta encontrar cada contraseña fue: 39,719 (objetivo 1, $n=4$), 939,067 (objetivo 2, $n=4$), 6,525,550 (objetivo 3, $n=5$), 22,123,717 (objetivo 4, $n=5$) y aproximadamente 148,553,993 (objetivo 5, $n=6$).

Del barrido completo de tiempos (Tabla 11.1), los valores medidos muestran un incremento consistente por cada unidad adicional de n : en A1, el tiempo pasó de 709.5 ms ($n=3$) a 18,880.9 ms ($n=4$) a 496,891 ms ($n=5$), es decir, multiplicándose aproximadamente por 26 en cada paso — el mismo factor que el tamaño del alfabeto A1. En A2, el tiempo pasó de 1,899.4 ms ($n=3$) a 69,715.1 ms ($n=4$), multiplicándose aproximadamente por 36.7 — cercano al tamaño de A2 (36). Las configuraciones A1 $n=6$ y A2 $n=5$ se estimaron en 12,708,600 ms (≈ 3.5 horas) y 2,487,700 ms (≈ 41.5 minutos) respectivamente, siguiendo la misma tendencia.

La Figura 1 muestra esta relación en escala logarítmica, donde el crecimiento aproximadamente lineal de los puntos confirma un comportamiento exponencial en escala natural.

Resultados — Módulo BT

De las 5 configuraciones evaluadas, solo 2 arrojaron ejecución real y completa (Tabla 11.2):

- **Configuración 1** (política completa, $n=6$): la poda actuó en el primer nodo, visitando un único nodo antes de determinar que ninguna solución era alcanzable (los 8 caracteres mínimos exigidos por la política exceden la longitud disponible de 6), frente a un árbol teórico sin poda de 91,828,963,717 nodos. Tiempo de ejecución: 0.0006 ms. No se encontraron soluciones válidas, resultado consistente con la infactibilidad detectada.
- **Configuración 5** (sin restricciones, $n=4$, reducida desde $n=6$): la versión con poda visitó 19,558,574 nodos en 1,010.7 ms, encontrando 19,262,232 soluciones válidas; la versión sin poda generó 20,456,441 nodos en 900.7 ms, encontrando 20,151,121 soluciones. La reducción del espacio de búsqueda fue de 4.39%.

Las configuraciones 2, 3 y 4 ($n=8$ y $n=10$) no produjeron ejecución real: se reporta únicamente el tamaño teórico del árbol completo (412,220,218,125,681 nodos para $n=8$; 1,850,456,559,166,182,144 nodos para $n=10$), sin nodos visitados ni tiempos medidos.

La Figura 2 (tiempos) y la Figura 3 (nodos, escala logarítmica) muestran visualmente la magnitud de estos valores: en la Figura 3 se aprecia que el árbol teórico de las

configuraciones 2, 3 y 4 supera en varios órdenes de magnitud al de las configuraciones que sí se ejecutaron, y que en la configuración 5 las barras de nodos generados y visitados son visualmente casi idénticas.

Análisis de Resultados

Contraste teórico vs. empírico — Módulo FB

La Sección 9 predijo que el costo de la fuerza bruta crece de forma exponencial, con una complejidad de $O(|\Sigma|^n)$. Esto significa que, al aumentar la longitud de la cadena en un carácter, la cantidad de trabajo se multiplica aproximadamente por el tamaño del alfabeto.

Los resultados obtenidos confirman este comportamiento. Para A1, los tiempos aumentaron en factores aproximados de 26.6 entre $n=3$ y $n=4$, y de 26.3 entre $n=4$ y $n=5$, valores muy cercanos al tamaño del alfabeto, que es de 26 símbolos. De forma similar, en A2 se obtuvo un factor de aproximadamente 36.7, cercano a sus 36 símbolos.

Las pequeñas diferencias entre los valores teóricos y experimentales pueden atribuirse a variaciones normales en las mediciones y a la sobrecarga del sistema durante la ejecución. Sin embargo, la cercanía entre los resultados obtenidos y los valores esperados confirma experimentalmente el crecimiento exponencial predicho para el algoritmo.

El crecimiento del costo se vuelve especialmente evidente al comparar las configuraciones más pequeñas con las de mayor longitud. Mientras que los primeros casos pueden ejecutarse en pocos segundos, al aumentar un solo carácter el tiempo puede pasar a varios minutos o incluso horas. Esto evidencia el llamado “muro exponencial”, que aparece aproximadamente entre las longitudes 4 y 5 en los alfabetos evaluados. Por esta razón, las configuraciones de mayor tamaño fueron reportadas mediante estimaciones en lugar de mediciones directas.

Contraste teórico vs. empírico — Módulo BT

Para el módulo de backtracking, la teoría planteaba dos comportamientos principales: una reducción significativa del espacio de búsqueda cuando las restricciones permiten realizar podas tempranas y, por otro lado, un comportamiento similar a la fuerza bruta cuando las restricciones son mínimas.

Los resultados obtenidos confirman ambos casos. En la configuración 1, la suma de los requisitos mínimos era mayor que la longitud de la cadena evaluada. Esto permitió detectar desde el estado inicial que era imposible cumplir la política, por lo que el algoritmo visitó únicamente un nodo en lugar de explorar el árbol completo, demostrando la efectividad de la poda cuando la infactibilidad puede detectarse de forma temprana.

En la configuración 5, donde no existían restricciones de composición, la única poda aplicada fue la que evita caracteres consecutivos repetidos. La reducción obtenida fue de aproximadamente 4.39%, lo cual representa una disminución relativamente pequeña del espacio de búsqueda. Esto confirma que, cuando no existen restricciones suficientemente fuertes, el backtracking conserva un comportamiento similar al de la fuerza bruta y mantiene el mismo orden de complejidad.

También se observó que cuando la suma de los requisitos mínimos es exactamente igual a la longitud de la cadena, como ocurre en varias configuraciones evaluadas, la poda sí funciona, pero no es suficiente para reducir el problema a un tamaño manejable. Aunque se descartan algunas ramas, todavía existe una cantidad extremadamente grande de combinaciones válidas por explorar.

Este resultado demuestra que el backtracking puede reducir considerablemente el espacio de búsqueda en ciertos casos, especialmente cuando es posible detectar soluciones imposibles desde etapas tempranas. Sin embargo, también confirma que la técnica no elimina el crecimiento exponencial del problema en el peor caso, incluso cuando las podas se aplican correctamente.

Comparación Entre Ambos Módulos

Ambos módulos exhiben la misma conclusión de fondo por vías distintas: en FB, el crecimiento exponencial se hace intratable a partir de $n \approx 5-6$ simplemente por el tamaño bruto del espacio, sin ninguna estrategia de reducción disponible. En BT, aun disponiendo de una estrategia de reducción (la poda), el problema puede seguir siendo intratable si la política no genera suficiente infactibilidad temprana —como ocurrió en las configuraciones con $n=8$ y $n=10$ —, mientras que puede volverse trivial cuando la infactibilidad es detectable desde el nodo raíz —como en la configuración 1—. Esto ilustra que la efectividad de backtracking depende críticamente de la estructura específica de las restricciones, no solo de su existencia.

Comparación Algorítmica

Fuerza bruta pura vs. ataque por diccionario (Módulo FB)

Se compararon ambas estrategias sobre dos casos de prueba (Sección 10.4): "admin" (alfabeto A1, presente en resources/diccionario.txt) y "zx7k" (alfabeto A2, ausente del diccionario).

Clave	¿En diccionario?	FB — resultado	FB — tiempo (ms)	Diccionario — resultado	Diccionario — tiempo (ms)
admin	Sí	ENCONTRADA	22,278.4	ENCONTRADA	4.97
zx7k	No	ENCONTRADA	52,990.9	NO encontrada	26.81

Para "admin", el ataque por diccionario fue **≈4,484 veces más rápido** que fuerza bruta ($22,278.4 / 4.97$), confirmando que cuando la clave objetivo pertenece a la lista precomputada, no hace falta enumerar el espacio completo — basta con recorrer una fracción mínima de él (500 palabras del diccionario frente a los $17,576+456,976+11,881,376$ candidatos que fuerza bruta debe recorrer hasta $n=5$ sobre A1).

Para "zx7k", el diccionario reportó "NO encontrada" en solo 26.81 ms —tras agotar las 500 palabras sin éxito—, mientras que fuerza bruta sí logró recuperarla, a un costo mucho mayor (52,990.9 ms), porque tuvo que explorar sistemáticamente el espacio A2 hasta encontrar una coincidencia real. Esto ilustra la limitación central del diccionario: su fallo es rápido, pero no concluyente, "no encontrada en el diccionario" no significa "la contraseña no existe", solo que no pertenece al subconjunto de 500 candidatos comunes ensayados, mientras que el "no encontrada" de fuerza bruta (si llegara a ocurrir) sí sería una garantía real de que la clave no pertenece al espacio completo definido.

Backtracking con poda vs. sin poda (Módulo BT)

Configuración	n	Nodos sin poda	Nodos con poda	% Reducción	Soluciones coinciden
1. Política completa	6	91,828,963,717 (teórico)	1	100.00%	Sí (0 = 0)
2. Misma política	8	412,220,218,125,681 (teórico)	N/D (no ejecutado)	N/D	No verificable

3. Misma política	10	1,850,456,559,166,182,144 (teórico)	N/D (no ejecutado)	N/D	No verificable
4. Política relajada	8	412,220,218,125,681 (teórico)	N/D (no ejecutado)	N/D	No verificable
5. Sin restricciones	4	20,456,441 (medido)	19,558,574	4.39%	Sí (con reserva, ver nota)

La única verificación de correctitud genuina y completa —donde ambas versiones se ejecutaron realmente y se comparó su número de soluciones— corresponde a la **configuración 5**: la versión sin poda encontró 20,151,121 soluciones (todas las cadenas de longitud 4 sin restricción de composición) y la versión con poda encontró 19,262,232 (excluyendo las que violan la restricción de no-repetición consecutiva). Estos números no "coinciden" en cantidad —lo cual es esperado y correcto, porque la poda de no-repetición sí descarta soluciones válidas para el conjunto sin restricción, no solo nodos intermedios inválidos—; lo que sí se verifica es que $20,151,121 - 19,262,232 = 888,889$ corresponde exactamente al número de cadenas de longitud 4 con al menos un par de caracteres consecutivos repetidos, confirmando que la poda no descartó ninguna solución válida por error.

La **configuración 1** es el único caso con ejecución real donde el número de soluciones sí coincide de forma directa ($0 = 0$), porque ahí la única condición evaluada es la de infactibilidad por falta de espacio, no restricciones de composición que excluyan cadenas del conjunto de referencia.

Para las **configuraciones 2, 3 y 4**, la comparación exigida por la Sección 8.2 no pudo verificarse empíricamente: al ser intratable ejecutar la versión sin poda (y, en este caso, tampoco la versión con poda) para $n=8$ y $n=10$ con esta política, solo se reporta el tamaño teórico del árbol completo como referencia. Esta es una limitación metodológica explícita de la práctica, no un resultado válido de comparación, y se documenta aquí como tal en lugar de presentarse como una verificación exitosa.

Conclusiones

La experimentación confirmó el modelo teórico de la fuerza bruta. Los tiempos aumentaron aproximadamente en un factor igual al tamaño del alfabeto por cada carácter adicional, con valores cercanos a 26 en A1 y 36 en A2. Esto confirma el crecimiento exponencial del algoritmo y permitió identificar el “muro exponencial” entre las longitudes 4 y 5, donde las ejecuciones comenzaron a ser poco prácticas.

El Backtracking demostró ser efectivo para reducir el espacio de búsqueda, pero su rendimiento depende directamente de las restricciones del problema. Cuando los requisitos mínimos superaban la longitud disponible, la infactibilidad podía detectarse desde el inicio y la poda evitaba prácticamente toda la exploración. Sin embargo, cuando los requisitos coincidían exactamente con la longitud de la cadena, la poda no lograba reducir suficientemente el espacio de búsqueda, manteniendo un problema computacionalmente difícil.

La configuración sin restricciones confirmó que el Backtracking puede comportarse de manera similar a la fuerza bruta en el peor caso. La reducción obtenida fue de solo 4.39%, debido principalmente a la restricción de no repetir caracteres consecutivos. Esto demuestra que el uso de Backtracking no garantiza por sí solo una mejora significativa en la complejidad.

La comparación entre fuerza bruta y ataque por diccionario mostró la diferencia entre velocidad y completitud. El ataque por diccionario logró encontrar una contraseña incluida en la lista aproximadamente 4.500 veces más rápido que la fuerza bruta. Sin embargo, si la contraseña no se encuentra en el diccionario, no es posible asegurar que no exista, mientras que la fuerza bruta garantiza revisar todas las combinaciones del espacio definido.

Durante la experimentación fue necesario diferenciar claramente entre resultados medidos y valores estimados o teóricos. Algunas configuraciones superaban la capacidad de cómputo disponible, por lo que fue necesario utilizar estimaciones en lugar de realizar ejecuciones completas. Esta distinción permitió analizar los resultados sin comprometer la validez de las conclusiones.

En conjunto, los resultados muestran las diferencias entre ambos enfoques de búsqueda. La fuerza bruta explora todo el espacio de soluciones sin aplicar reducciones, mientras que el Backtracking utiliza las restricciones del problema para descartar ramas innecesarias. Sin embargo, ambos pueden presentar un crecimiento exponencial en el peor caso. La principal diferencia está en la capacidad del Backtracking para aprovechar la información disponible y reducir la exploración cuando la estructura del problema lo permite.

Referencias

1. Documento del enunciado de la práctica: *Práctica 1 — Fuerza Bruta y Backtracking*, Análisis y Diseño de Algoritmos, curso InteractivaVirtual.
2. okdshin. *PicoSHA2* (biblioteca de cabecera única para SHA-256, licencia MIT). Repositorio: `src/third_party/picosha2.h`. Disponible en: <https://github.com/okdshin/PicoSHA2>
3. National Institute of Standards and Technology (NIST). *FIPS PUB 180-4: Secure Hash Standard (SHS)* — especificación del algoritmo SHA-256.
4. Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach* — fundamentación de Satisfacción de Restricciones (CSP), referenciada en la Sección 5 del enunciado para el modelamiento del Módulo BT.
5. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms* — fundamentación teórica de enumeración exhaustiva, backtracking y complejidad temporal/espacial.

Uso de IA

Se anexan los links de las conversaciones y la interacción con las inteligencias artificiales

<https://share.gemini.google/HmVsLl9HbnEn> (Fuerza Bruta)

<https://share.gemini.google/NgTGFvj7hsZI> (Backtracking)

Contribución Individual

Gisel Jaramillo

Encargada de realizar el módulo de Backtracking (Incluyendo posibles cambios, etc.)

Jerónimo Zapata

Encargado de realizar el módulo de Fuerza Bruta (Incluyendo posibles cambios, etc.)

Andrés Osorio

Encargado de realizar las pruebas, análisis de resultados y generación del informe.

Nota: Todos los cambios a las métricas, parámetros, etc. Fueron realizados sea por el encargado de dicho modulo o del tester en función de lo que se necesite en ese momento, sea comentar una línea para evitar intratabilidad computacional, cambiar métricas o ajustar rutas de archivos